

Actualization — SOLID, Clean Architecture & Clean Code

CASE study: JHotDraw 7 | **Feature:** Group / Ungroup figures

Actualization is the phase where new functionality is implemented, incorporated into the old code, and its change is propagated to every place that needs a secondary modification. SOLID, Clean Architecture and Clean Code are the principles that keep that incorporation and propagation safe and localized. The examples below are taken from the classes located for the Group/Ungroup feature.

1. SOLID principles in the CASE study

S — Single Responsibility Principle (SRP)

A class should have one reason to change. JHotDraw separates the responsibilities of the feature across distinct classes: `Drawing` (the model that owns figures), `DrawingView` (selection + display), `DrawingEditor` (coordinates views/tools) and `GroupAction` (the controller that starts grouping). Each changes for one reason.

- **Violation** → **fix:** `GroupAction` carries an `isGroupingAction` boolean that makes one class do two jobs (group **and** ungroup) — an SRP smell. The SRP-respecting design is the separate `UngroupAction` class, and the L4 refactoring (Compose Method) split the work into `groupSelectedFigures()` and `ungroupSelectedFigures()` so each method has a single responsibility.

O — Open/Closed Principle (OCP)

Open for extension, closed for modification. New figure types are added by extending the `Figure` / `AbstractFigure` abstraction — `GroupFigure`, `RectangleFigure`, `EllipseFigure`, etc. — **without modifying** `Drawing`, `DrawingView` or the grouping logic, because they all operate on the `Figure` abstraction. Likewise new commands extend `AbstractSelectedAction` without changing the action framework. Grouping a brand new figure type requires zero changes to `GroupAction`.

L — Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base type. `GroupFigure` **is-a** `CompositeFigure` **is-a** `Figure`. After grouping, the `Drawing` and `DrawingView` treat the resulting `GroupFigure` exactly like any other `Figure` — it can be drawn, selected, moved and transformed through the same `Figure` interface. The whole point of the feature relies on LSP: a group of figures can stand in wherever a single figure can. Similarly `UngroupAction` extends `GroupAction` and is usable wherever a `GroupAction` is expected.

I — Interface Segregation Principle (ISP)

Clients should not depend on methods they do not use. JHotDraw favours many small, focused interfaces rather than one fat one: `Figure` , `CompositeFigure` , `ConnectionFigure` , `DOMStorable` (persistence), `Disposable` (lifecycle), `Handle` , and the listener interfaces `FigureListener` , `CompositeFigureListener` , `FigureSelectionListener` . A class implements only what it needs — e.g. `AbstractSelectedAction` implements `Disposable` , and `AbstractCompositeFigure` listens to its children through a `FigureAdapter` (default implementation of `FigureListener`) instead of being forced to implement unrelated methods.

D — Dependency Inversion Principle (DIP)

Depend on abstractions, not concretions. `GroupAction` imports and depends only on the **interfaces** `DrawingEditor` , `DrawingView` , `Drawing` , `CompositeFigure` and `Figure` — never on concrete classes such as `DefaultDrawing` or `DefaultDrawingView` . The composite used for grouping is injected through the constructor as a `CompositeFigure` *prototype*, so the high-level grouping policy is decoupled from any concrete group implementation. The high-level action does not depend on low-level figure details; both depend on the `Figure` abstraction.

2. Clean Architecture in the CASE study

Clean Architecture organizes code into concentric layers with **The Dependency Rule**: source-code dependencies point only **inward**; inner layers know nothing about outer ones. Mapping the Group/Ungroup feature onto the layers:

Layer (inner → outer)	Group/Ungroup classes	Role
Entities (enterprise rules)	<code>Figure</code> , <code>CompositeFigure</code> , <code>GroupFigure</code> , <code>Drawing</code>	The core domain: what a figure / group / drawing is and how children are contained. Knows nothing about Swing.
Use Cases (application rules)	the <code>groupFigures()</code> / <code>ungroupFigures()</code> operations + the <code>UndoableEdit</code> orchestration	The application-specific rule: "group the selected figures into one (undoably)."
Interface Adapters	<code>GroupAction</code> , <code>UngroupAction</code> , <code>DrawingView</code>	Translate a Swing menu event into model operations and adapt the model for display/selection.
Frameworks & Drivers	Swing (<code>AbstractAction</code> , <code>ActionEvent</code>), <code>javax.swing.undo</code> , the GUI, resource bundles	The volatile outer details.

Dependency Rule in action: the inner `Figure` / `Drawing` model has no reference to `GroupAction` or to Swing; it is the action (an outer adapter) that depends inward on the model abstractions. This is why the model can be reused in a different UI without change — exactly what makes actualization (incorporating new behaviour) safe.

Where JHotDraw deviates (and what Clean Architecture would do): `GroupAction` is both a controller *and* the home of the grouping use-case logic, and it reaches directly into `javax.swing.undo` . A stricter Clean Architecture

would **extract the use case** (a `GroupFiguresInteractor`) behind a boundary interface, leaving `GroupAction` as a thin controller. That separation would let the grouping rule be tested and reused independently of Swing.

3. Clean Code principles in context of Actualization

Actualization adds and propagates change; Clean Code keeps that change readable and local:

- **Meaningful names / intention-revealing methods** — the L4 refactoring replaced a 69-line `actionPerformed` with `groupSelectedFigures()` / `ungroupSelectedFigures()`, so a reader sees *what* happens, not *how*.
- **Small functions, one level of abstraction** — Compose Method gives each method a single job, the cure applied during the refactoring lab.
- **DRY (Don't Repeat Yourself)** — the duplicated `DrawLabels.getLabels()...` lookup was extracted into `labelText(key)`.
- **Command-Query Separation** — `canGroup()` / `canUngroup()` are queries (no side effects) used to guard the commands `groupFigures()` / `ungroupFigures()`.

Together, SOLID + Clean Architecture + Clean Code mean a secondary modification during actualization (e.g. supporting a new figure type in a group) propagates to few, well-isolated places instead of rippling through the whole codebase.